

Full Stack Development with MERN

DARSHAN EASE

A Full Stack Web Application for Smart Temple Darshan Management

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING

Submitted by

LAXMIKANTH LALAM (TEAM LEAD)

MEDISETTI SANTOSH

MEERZA KANEEZ KULSUM

SHANKARA MANI SAI NOOKALAMANTHI

AKHILA PRIYA NOOKARAPU

ACKNOWLEDGEMENT

We express our sincere gratitude to [APSCHE-SMARTBRIDGE](#) for providing us with the opportunity to undertake the Full Stack Development project titled [DARSHAN EASE](#).

This project enabled us to gain practical experience in designing, developing, and deploying a modern MERN-stack web application. Throughout the project, we enhanced our understanding of React, Node.js, Express.js, MongoDB, REST APIs, authentication mechanisms, and full-stack application deployment.

We are also thankful to our faculty members and everyone who directly or indirectly contributed by offering suggestions, encouragement, and constructive feedback during the development process.

Finally, we thank our team members for their dedication, cooperation, and commitment, which made the successful completion of this project possible.

ABSTRACT

DARSHAN EASE is a web-based temple management and online darshan booking system developed using the MERN (MongoDB, Express.js, React.js, and Node.js) technology stack. The application simplifies the process of booking temple darshan by allowing devotees to register, authenticate securely, select available dates and time slots, and manage their bookings through an intuitive user interface.

The system incorporates secure user authentication using JSON Web Tokens (JWT) and password encryption through encrypt, ensuring the protection of user information. Administrative users can efficiently manage temple schedules, booking requests, and user records using dedicated dashboards.

The frontend is built with React and Vite to provide a responsive and interactive user experience, while the backend exposes RESTful APIs using Express.js. MongoDB serves as the primary database for storing user profiles, booking information, and application data.

The application demonstrates modern full-stack development practices, including REST API integration, responsive UI design, authentication, database management, and cloud deployment. DARSHAN EASE aims to reduce manual effort, improve transparency, minimize waiting times, and provide devotees with a convenient digital platform for temple darshan management.

1. Introduction

Introducing Darshan Ease – your seamless digital gateway to spiritual journeys and divine experiences! Navigating temple visits, scheduling holy darshans, and planning spiritual pilgrimages can often be overwhelming due to crowded booking portals and complex schedules. Our application is designed to simplify your spiritual path by bringing hassle-free booking, transparent queuing tracking, and multi-temple itineraries directly to your fingertips. Whether you are seeking a special solo darshan, arranging an elaborate family pilgrimage, or securing ritual slots (Sevas), Darshan Ease ensures that your booking workflow is smooth, transparent, and respectful of your time. Embark on a stress-free journey to devotion with Darshan Ease – where every milestone brings you closer to peace!

2. Project Overview

Purpose:

The Darshan Ease application is developed to provide a modern, secure, and user-friendly online platform for booking and managing religious darshans and allied spiritual services. The project aims to eliminate the friction from religious tourism by offering real-time slot selection across various temples and timing windows. Built using the MERN Stack (MongoDB, Express.js, React.js, and Node.js), the application enables devotees to register securely, search for slot availabilities, configure a booking checkout, place reservations, and view historical bookings. It also provides dedicated management interfaces for temple trust representatives and administrators to update slot schedules, monitor crowd distribution, check verified tickets, and maintain records efficiently, ensuring a seamless experience for all stakeholders.

Features:

The Darshan Ease Application provides the following key features:

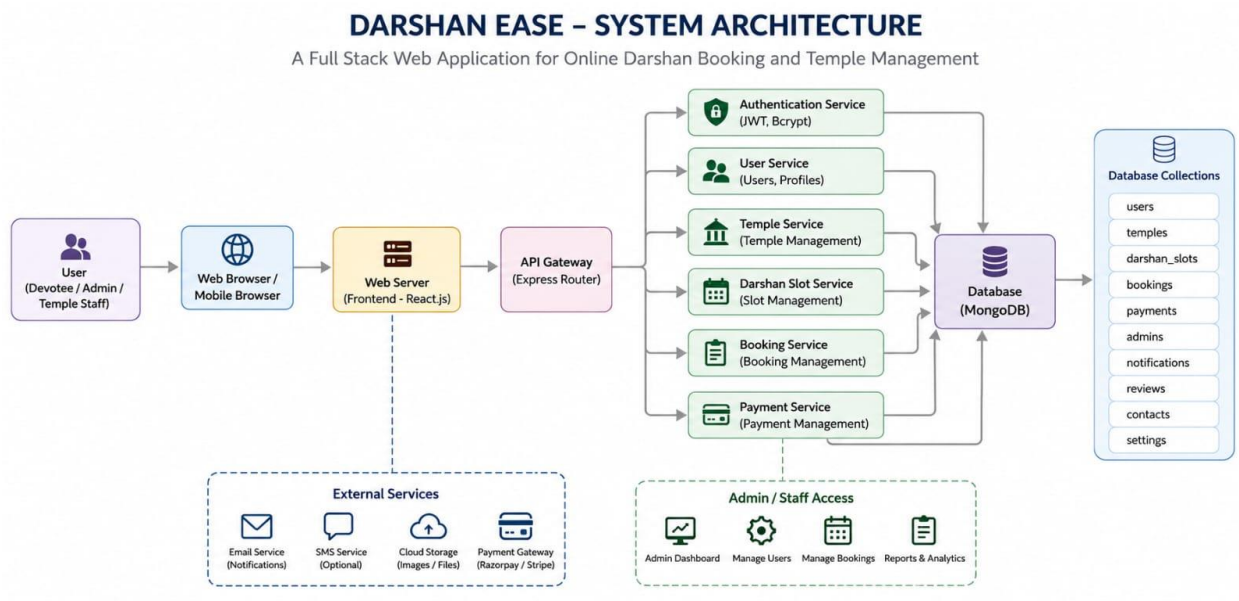
- **User Registration and Authentication:** Secure account registration, profile creation, and user authentication.
- **Temple and Slot Listings:** Explore detailed temple directories with active slot structures, pricing guidelines, age/identity constraints, and operational timings.
- **Advanced Availability Search:** Search and filter active darshans and custom Sevas by date, time window, temple location, and general popularity.

- Devotee Allocation & Slot Checkout: Multi-person add-on selectors to attach pilgrim passenger information and aggregate sub-tickets before checking out.
- Secure Reservation Engine: Reserve slots securely with dynamic concurrency protection, ensuring no double-booking occurs during peak operational periods.
- Instant Pass Generation: Generate dynamic bookings featuring a unique digital confirmation receipt holding verification tokens.
- Booking Dashboard & History: View past experiences, download upcoming travel passes, track reservation status, and submit cancellations when necessary.
- Temple Operator/Trustee Portal: Dedicated layouts allowing temple authorities to add custom slots, update maximum capacities, check ticket validations, and edit ritual details.
- Admin Dashboard: Platform-wide oversight to audit user balances, manage associated temple registrations, inspect transaction history, and overview overall platform performance metrics.
- Capacity Control & Inventory Management: Monitor and throttle slot capacities dynamically based on real-world festival conditions or crowd control parameters.
- Reporting and Analytics: Generate tabular insights on peak booking days, popular temple slots, user demographic metrics, and platform usage rates.
- Responsive User Interface: Provides an intuitive and consistent experience across desktop, tablet, and mobile browsers.
- RESTful API Integration: Efficient communication layers between React components, backend Express logical route points, and the MongoDB engine.
- Scalable MERN Architecture: Modular design built from the ground up for high horizontal data scalability and minimal API latency.

Future Enhancements:

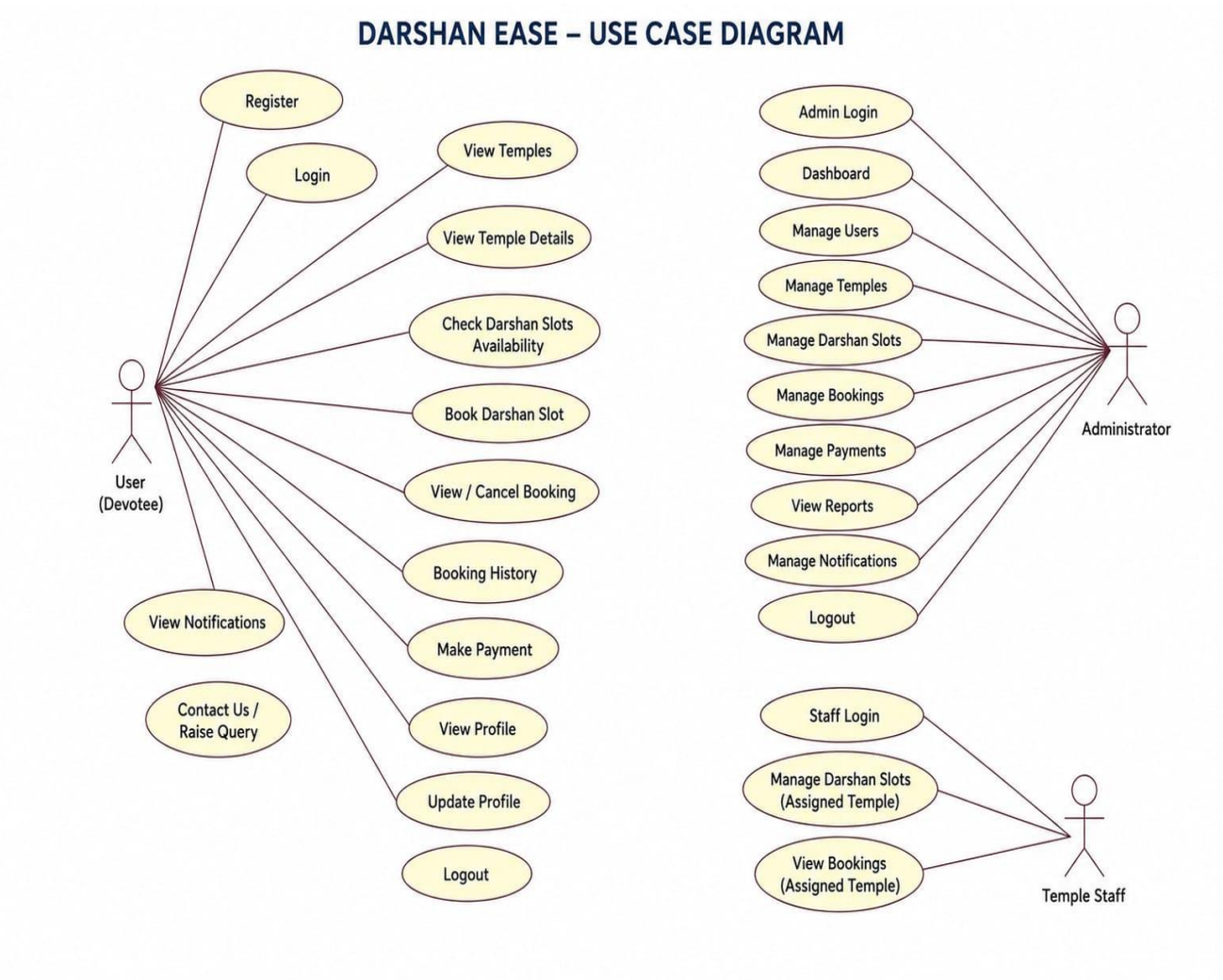
- Integration with official payment gateways (Stripe/Razorpay) for premium prasadam and ritual transactions.
- GPS-enabled real-time queue delay trackers and predictive AI-based crowd forecasting engines.
- Cloud storage integration for physical government ID storage checks.

3. Architecture



- **Frontend:** The UI layer is implemented in React.js leveraging a decoupled layout architecture. Routing rules are maintained through React Router DOM, handling conditional states for Devotees, Temple Trusts, and Platform Admins. API state syncing is streamlined using modern network clients (Axios) to bind UI views to server resources efficiently.
- **Backend:** Developed atop Node.js and Express.js using a modular REST architecture. It utilizes specific request middleware configurations (CORS, body parsing routing) alongside JSON Web Token validation layers to shield administrative resources from unauthenticated requests.
- **Database:** A flexible document-based paradigm utilizing MongoDB and Mongoose ODM. Schemas are explicitly typed to support atomic reservation increments, state validations, and relational mapping pointers between primary collections.

4. ER Diagram (Entity-Relationship)



5. Setup Instructions

Prerequisites:

Before configuring Darshan Ease, ensure the following applications exist in your development container:

Software Requirements

- Node.js (Latest LTS Version)
- npm (Node Package Manager)
- MongoDB Community Edition or MongoDB Atlas
- Git

- Visual Studio Code (or any preferred IDE)

Frontend Dependencies

- React.js
- React Router DOM
- Axios
- Bootstrap
- React-Bootstrap

Backend Dependencies

- Express.js
- Mongoose
- CORS
- dotenv
- JSON Web Token (JWT)
- bcrypt/bcryptjs
- Body-parser

Installation Steps:

1. Clone the repository:

```
git clone https://github.com/your-username/darshan-ease.git
```

2. Step inside the primary folder base:

```
cd darshan-ease
```

3. Install web client interface dependencies:

```
cd client && npm install
```

4. Install logical service backend components:

```
cd ../server && npm install
```

5. Create environment variable parameters (.env in /server):

```
PORT=5000
```

```
MONGO_URI=your_mongodb_connection_string
```

```
JWT_SECRET=your_spiritual_app_secret_hash
```


6. Folder Structure

Client Structure:

darshan-ease/ client/

```
├── README.md
├── index.html
├── package.json
├── vite.config.js
├── .gitignore
├── public/
│   ├── favicon.svg
│   └── icons.svg
└── src/
    ├── App.jsx
    ├── index.css
    ├── main.jsx
    ├── assets/
    │   ├── hero.png
    │   ├── react.svg
    │   └── vite.svg
    ├── components/
    │   ├── Footer.jsx
    │   ├── Loading.jsx
    │   ├── Navbar.jsx
    │   ├── TempleCard.jsx
    │   └── admin/
    │       ├── DashboardCard.jsx
    │       ├── Sidebar.jsx
    │       └── Topbar.jsx
    ├── context/
    │   └── AuthContext.jsx
    ├── layouts/
    │   └── AdminLayout.jsx
    ├── pages/
    │   └── Booking.jsx
```

```

|   |— Dashboard.jsx
|   |— Home.jsx
|   |— Login.jsx
|   |— MyBookings.jsx
|   |— NotFound.jsx
|   |— Profile.jsx
|   |— Register.jsx
|   |— TempleDetails.jsx
|   |— Temples.jsx
|   |— admin/
|       |— AddSlot.jsx
|       |— AddTemple.jsx
|       |— Bookings.jsx
|       |— Dashboard.jsx
|       |— EditSlot.jsx
|       |— EditTemple.jsx
|       |— Slots.jsx
|       |— Temples.jsx
|— routes/
|   |— AdminRoute.jsx
|   |— AppRoutes.jsx
|   |— GuestRoute.jsx
|   |— PrivateRoute.jsx
|— services/
|   |— api.js
|— utils/
    |— generateTicket.js

```

- **assets/**: App-specific iconography and temple banner imagery
- **components/**: Reusable atomic views: Navbar, Footer, TicketCard, SlotSelector
- **pages/**: Full page layout instances: Home, Login, Booking, Profile, Dashboard
- **context/**: State containers handling global Session/Auth tokens
- **services/**: Network layers managing target Axios backend paths
- **routers/**: Authorization-aware structural route rules
- **App.jsx/**: Root app mounting point

- **main.jsx/**: Bundler configuration file

Server Structure:

darshan-ease/ server/

```
|— app.js
|— package.json
|— server.js
|— .gitignore
|— config/
|   |— db.js
|— controllers/
|   |— admin.controller.js
|   |— auth.controller.js
|   |— booking.controller.js
|   |— slot.controller.js
|   |— stats.controller.js
|   |— temple.controller.js
|   |— user.controller.js
|— middleware/
|   |— admin.middleware.js
|   |— auth.middleware.js
|— models/
|   |— booking.model.js
|   |— slot.model.js
|   |— temple.model.js
|   |— user.model.js
|— routes/
|   |— admin.routes.js
|   |— auth.routes.js
|   |— booking.routes.js
|   |— slot.routes.js
|   |— stats.routes.js
|   |— temple.routes.js
|   |— user.routes.js
```

```
├── services/
│   ├── admin.service.js
│   ├── auth.service.js
│   ├── booking.service.js
│   ├── slot.service.js
│   ├── stats.service.js
│   ├── temple.service.js
│   └── user.service.js
├── utils/
│   └── generateToken.js
└── validations/
    ├── auth.validation.js
    ├── slot.validation.js
    ├── temple.validation.js
    └── user.validation.js
```

Description

- **config/** – Database connection configuration.
- **controllers/** – Business logic for API requests.
- **middleware/** – Authentication, authorization, and error-handling middleware.
- **models/** – Mongoose schemas and models.
- **routes/** – Express route definitions.
- **services/** – Service layer for business operations.
- **utils/** – Utility functions and helper methods.
- **index.js** – Entry point of the backend application.

7. Running the Application

Backend: Start the Backend Server

Step1: Navigate to the server directory:

```
cd server
```

Step2: Install dependencies (if not already installed):

npm install

Step3: Start the backend server:

npm start

Step4: The backend server will run on:

http://localhost:5000

Frontend: Start the Frontend Server

Step1: Navigate to the client directory:

cd client

Step2: Install dependencies:

npm install

Step3: For Create React App:

npm start

Step4: For Vite:

npm run dev

Step5: The frontend application will run on:

http://localhost:3000

or

<http://localhost:5173>

8. API Documentation

- POST /api/auth/register - Registers a new account context on the framework (Guest / Public)
- POST /api/auth/login - Validates target logins, returning a signed token vector (Guest / Public)
- GET /api/temples - Returns a filtered array of active holy sites (Registered Devotee)
- GET /api/slots/:templeId - Returns available timeslots for a specific temple (Registered Devotee)
- POST /api/slots - Inserts a new slot block layout entry (Temple Trust / Admin)
- PUT /api/slots/:id - Adjusts operational parameters or active slot capacities (Temple Trust / Admin)
- DELETE /api/slots/:id - Purges specific timeslots from active indices (Temple Trust / Admin)
- POST /api/bookings - Evaluates availability constraints and locks a reservation (Registered Devotee)
- GET /api/bookings - Fetches active bookings bound to the current credential header (Registered Devotee / Trustee)

9. Authentication & Authorization Policies

- Hashed Credential Integrity: Device accounts are secured at registration via a standard cryptographic one-way hashing function (bcryptjs), isolating plain text formats from application records.
- Stateless Token Propagation: Valid user requests require an active, securely-signed Bearer string passed via the HTTP Authorization request header.
- Role-Based Domain Access Controls (RBAC):
 - Devotee: Authorized to view lists, query structural availabilities, book tokens, and observe their own structural history logs.
 - Temple Trustee: Retains specialized access hooks enabling slot configurations, inventory tuning, and validating physical ticket credentials at access terminals.
 - System Administrator: Complete read/write access over active users, metadata configurations, logs, and billing parameters.

10. Testing

Testing was conducted to verify that all modules of the DARSHAN EASE application function correctly and meet the specified requirements. The application was tested for user registration, login, JWT-based authentication, darshan booking, profile management, and booking history. Integration testing ensured smooth communication between the React frontend, Express backend, and MongoDB database. Functional testing confirmed that each feature operated as expected, while security testing verified password encryption using bcrypt and secure access through JWT authentication. Browser compatibility testing was also performed on Google Chrome, Microsoft Edge, and Mozilla Firefox to ensure consistent performance. The testing results showed that all major functionalities worked successfully, the system was secure and reliable, and the application was ready for deployment.

11. Project Flow:

Before starting to work on this project, let's see the demo.

Project demo: https://drive.google.com/file/d/1lLatlt-gEKu0voJ_OQuXkTtZm97_kQCY/view?usp=sharing

Use the code in: <https://github.com/laxmikanthlalam/DarshanEase>

Milestone 1: Project setup and configuration.

- **Folder setup:**
 1. Create frontend and
 2. Backend folders
- **Installation of required tools:**
 1. Open the frontend folder to install necessary tools
 2. For frontend, we use:
 - React
 - Bootstrap
 - Material UI
 - Axios
 - Moment
 - Antd

- mdb-react-ui-kit
- react-bootstrap

3.Open the backend folder to install necessary tool

4.For backend, we use:

- cors
- bcryptjs
- express
- dotenv
- mongoose
- Moment
- Multer
- Nodemon
- jsonwebtoken

After the installation of all the libraries, the package.json files for the frontend looks like the one mentioned below.

```

1  package.json ! X
client > {} package.json > ...
2  {
3    "name": "client",
4    "private": true,
5    "version": "0.0.0",
6    "type": "module",
7    "scripts": {
8      "dev": "vite",
9      "build": "vite build",
10     "lint": "eslint",
11     "preview": "vite preview"
12   },
13   "dependencies": {
14     "axios": "^1.18.1",
15     "bootstrap": "^5.3.8",
16     "chart.js": "^4.5.1",
17     "jspdf": "^4.2.1",
18     "react": "^19.2.7",
19     "react-bootstrap": "^2.10.10",
20     "react-chartjs-2": "^5.3.1",
21     "react-dom": "^19.2.7",
22     "react-icons": "^5.7.0",
23     "react-router-dom": "^7.18.0",
24     "react-toastify": "^11.1.0",
25     "sweetalert2": "^11.26.25"
26   },
27   "devDependencies": {
28     "@types/react": "^19.2.17",
29     "@types/react-dom": "^19.2.3",
30     "@vitejs/plugin-react": "^6.0.2",
31     "eslint": "^1.69.0",
32     "vite": "^8.1.0"
33   }
34 }

```

After the installation of all the libraries, the package.json files for the backend looks like the one mentioned below.



```

package.json > {} dependencies
{
  "name": "server",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "bcryptjs": "^3.0.3",
    "cors": "^2.8.6",
    "dotenv": "^17.4.2",
    "express": "^5.2.1",
    "jsonwebtoken": "^9.0.3",
    "mongoose": "^9.7.2",
    "morgan": "^1.11.0",
    "validator": "^13.15.35"
  },
  "devDependencies": {
    "nodemon": "^3.1.14"
  }
}

```

Milestone 2: Backend Development

- **Setup express server**
 1. Create index.js file in the server (backend folder).
 2. define port number, mongodb connection string and JWT key in env file to access it.
 3. Configure the server by adding cors, body-parser.
- **Configure MongoDB**
 1. Import mongoose.
 2. Add database connection from config.js file present in config folder
 3. Create a model folder to store all the DB schemas like Booking,Payment, owner and booking, properties schemas.
- **Add authentication:** for this,

You need to make a middleware folder and in that make authMiddleware.js file for the authentication of the projects and can use it.

Milestone 3: Database Development:

- Set up a MongoDB database either locally or using a cloud-based MongoDB service like MongoDB Atlas.
- Create a database and define the necessary collections for users, transactions, payments and Booking slots.
- Also let's see the detailed description for the schemas used in the database.

Milestone 4: Frontend Development

1. Setup React Application

The **Darshan Ease** frontend was developed using **React.js** to create a fast, responsive, and interactive user interface. The development process began by setting up the React application, installing all the required dependencies, and organizing the project structure for efficient development.

The following activities were performed:

- Created the React application.
- Installed the required libraries (React Router, Axios, Redux Toolkit, Tailwind CSS, Swiper, Firebase, etc.).
- Created the project folder structure.
- Configured routing for navigation between pages.
- Organized reusable components, pages, assets, and utility files.

2. Design UI Components

The user interface was designed using reusable React components to provide a clean and user-friendly experience.

The following components were developed:

- Navigation Bar
- Home Page
- Dashboard Cards
- Sidebar
- Loading
- User Login and Registration
- Admin Dashboard
- TempleCard
- Footer
- Search Bar
- Category Filter

Responsive layouts and consistent styling were implemented using **Tailwind CSS** to ensure compatibility across different screen sizes.

3. Implement Frontend Logic

The frontend logic connects the user interface with the backend services and manages application data efficiently.

The implementation includes:

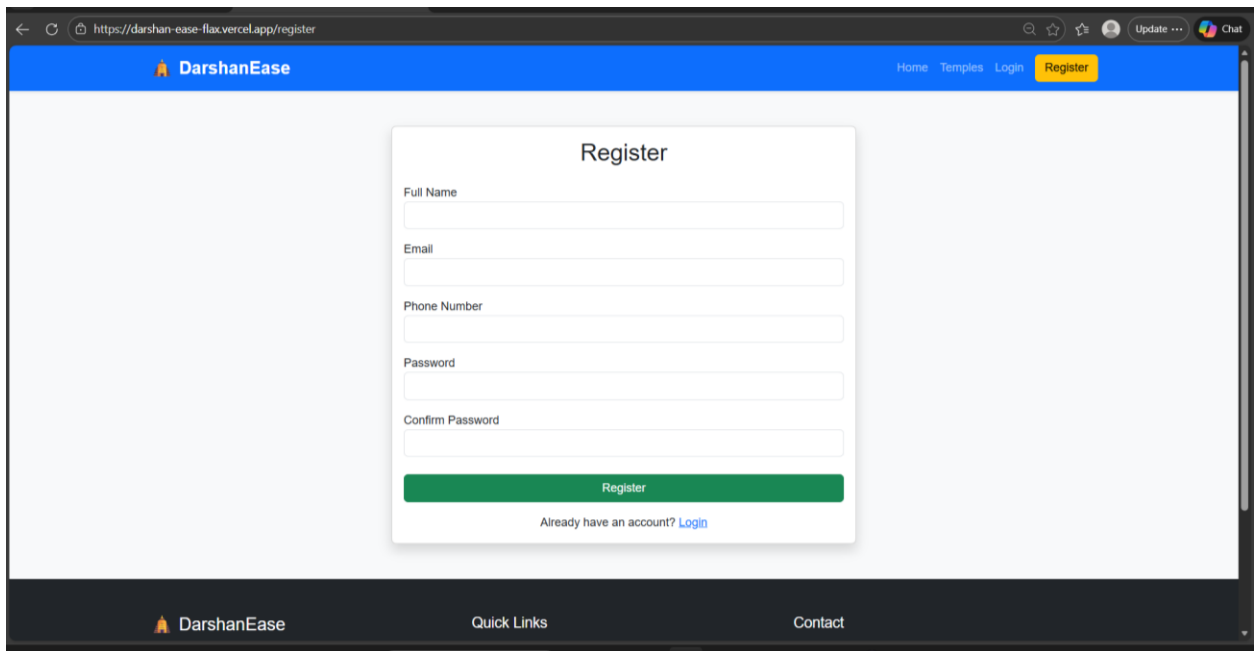
- Integration with backend REST APIs.
- User authentication.
- Fetching and displaying Temples from the database.
- Temples search and filtering.
- Slot Booking management.
- State management using Redux.
- Dynamic rendering of Temple information.

4.Database Schema

The Darhan Ease application uses **MongoDB** as the database and **Mongoose** as the Object Data Modeling (ODM) library. Mongoose schemas define the structure of the data stored in each collection.

12. Project Implementation Steps

- **User Registration (Sign Up):** The Registration module functions as the initial collection vector for devotees accessing the portal platform.



The screenshot shows the 'Register' page of the DarshanEase application. The page has a blue header with the DarshanEase logo and navigation links: Home, Temples, Login, and a yellow Register button. The main content area is light gray and contains a white registration form. The form has the title 'Register' and five input fields: Full Name, Email, Phone Number, Password, and Confirm Password. Below the fields is a green Register button. At the bottom of the form, there is a link: 'Already have an account? [Login](#)'. The footer is dark gray and contains the DarshanEase logo, Quick Links, and Contact information.

Register

Full Name

Email

Phone Number

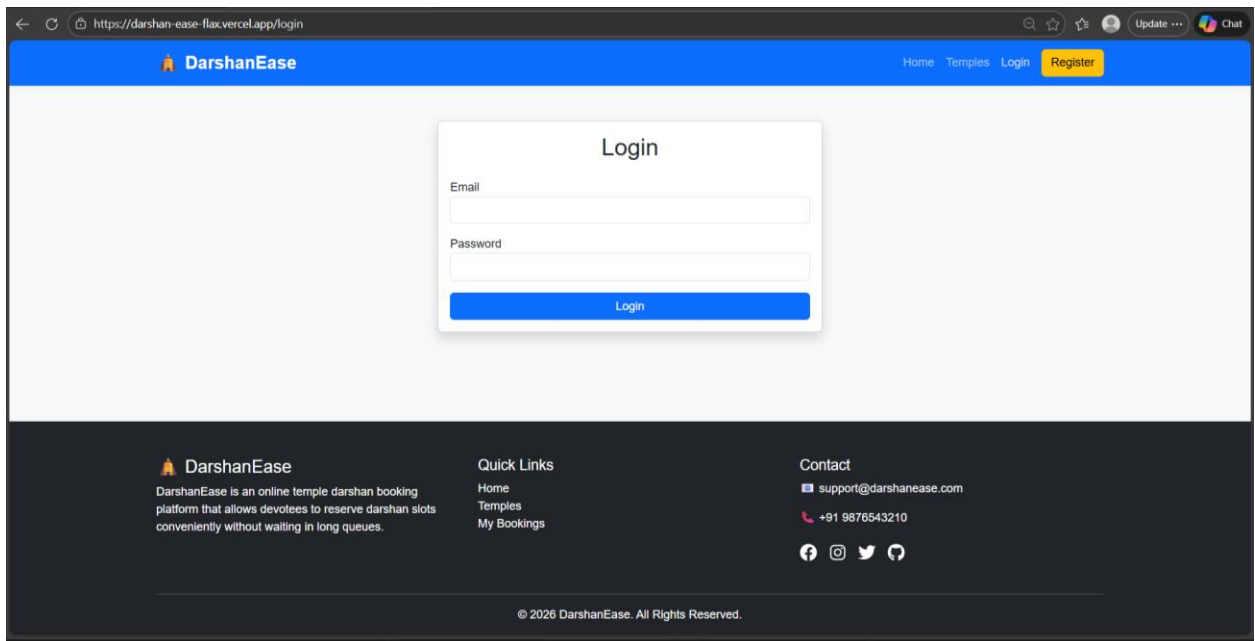
Password

Confirm Password

Register

Already have an account? [Login](#)

- **User Login:** The User Identity Validation checkpoint guarantees secure system initialization by challenging existing devotee profiles.



The screenshot shows the 'Login' page of the DarshanEase application. The page has a blue header with the DarshanEase logo and navigation links: Home, Temples, Login, and a yellow Register button. The main content area is light gray and contains a white login form. The form has the title 'Login' and two input fields: Email and Password. Below the fields is a blue Login button. The footer is dark gray and contains the DarshanEase logo, Quick Links, and Contact information.

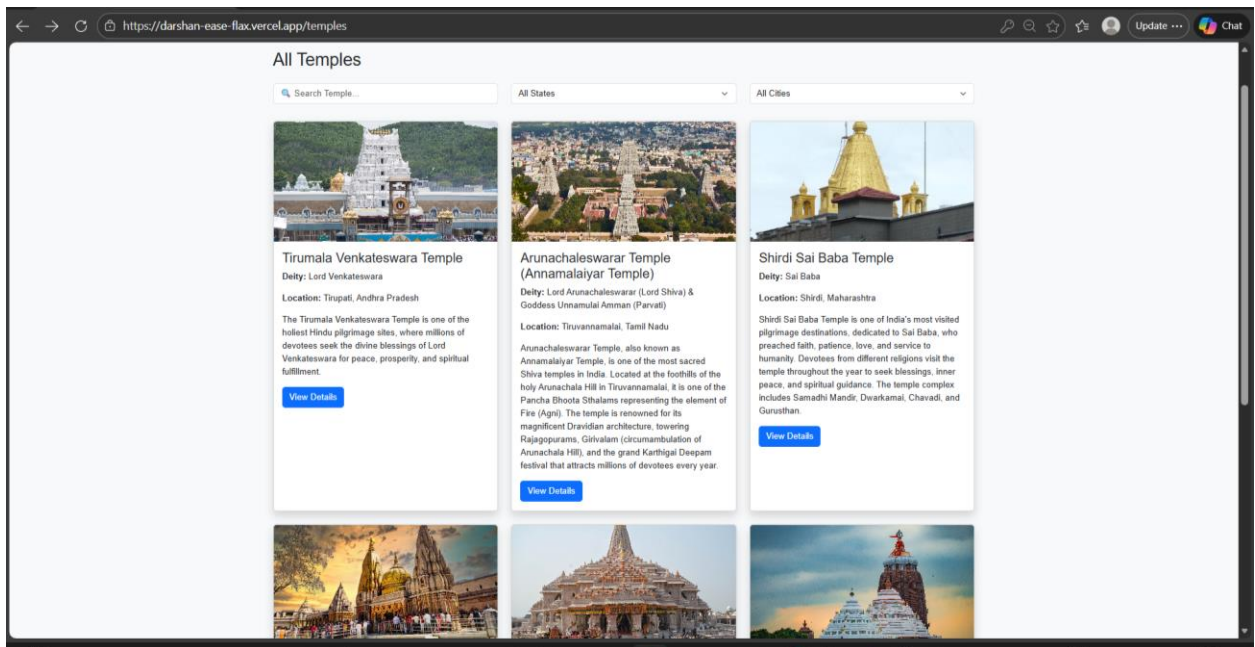
Login

Email

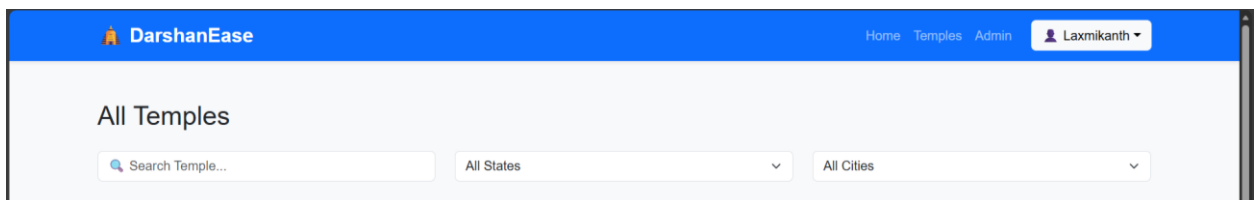
Password

Login

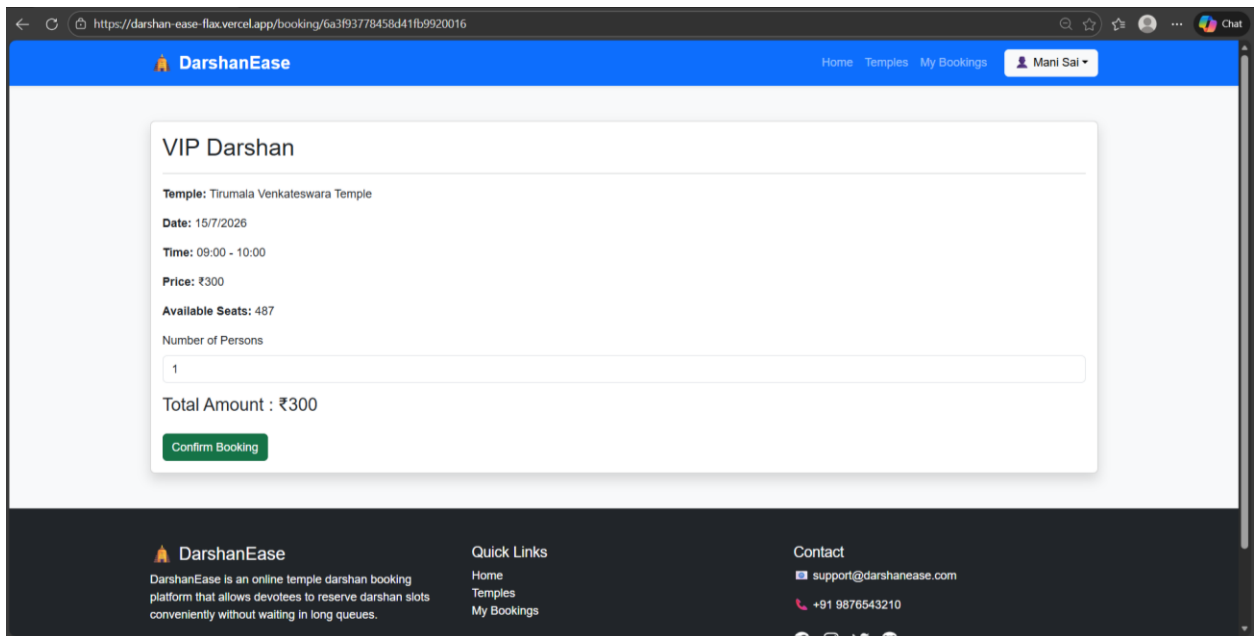
- **Browse Temples and Slots:** The central discovery directory compiles and renders active structural components mapping out all currently registered religious centers.



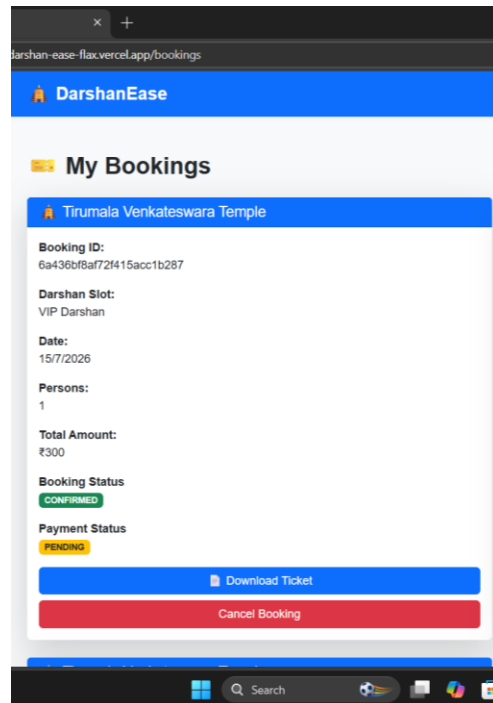
- **Search and Filter Engine:** To streamline travel planning, the interactive scheduling query bar lets user groups isolate distinct temple sites.



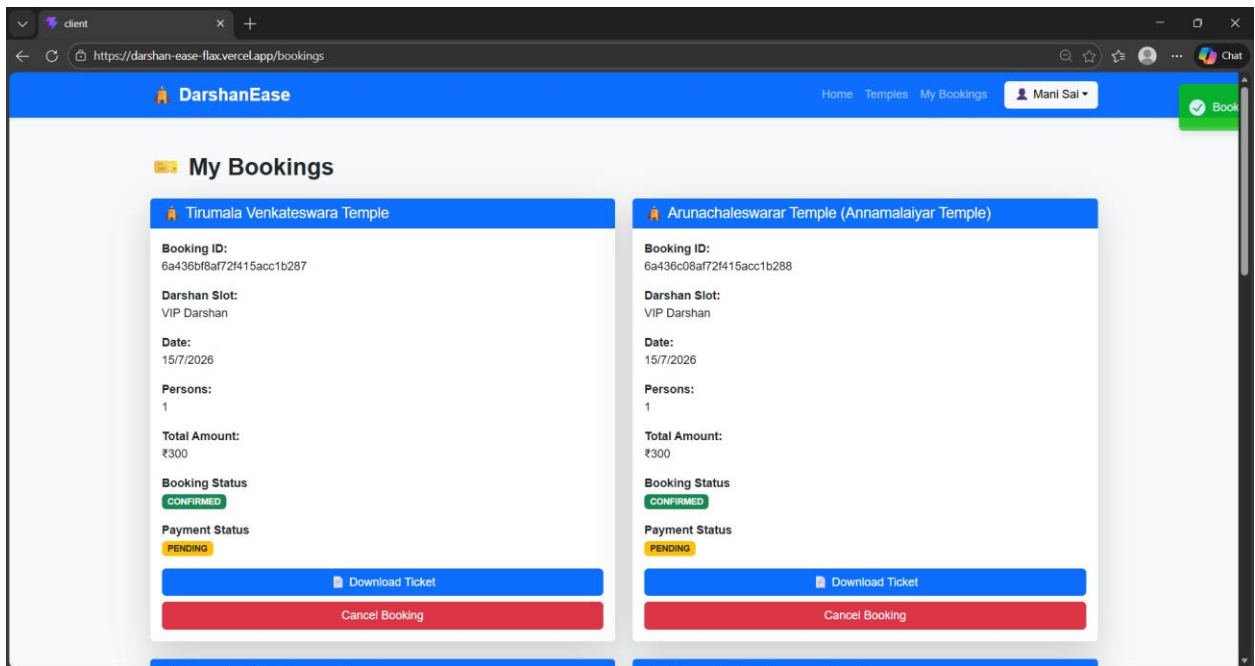
- **Slot Selection & Pilgrim Detail Form:** Before advancing to checkout pools, booking paths require specific data input detailing each individual passenger node.



- **Booking Confirmation and Receipt Checkout:** Once internal capacity locks ensure slot availability, the user steps into the confirmation interface.



- **Devotee Booking History Panel:** A dedicated secure dashboard context displays the user's booking history.



- **Temple Admin & Trust Management Dashboard:** The structural operator center grants authorized trustees comprehensive control over their physical facility parameters.

